

FlexForge — Technical Documentation

Runtime-interpretation no-code application platform · v0.1.0 · 2026-06-24

1. Executive Overview

FlexForge is a packaged runtime-interpretation engine: one hardened Spring Boot image reads an immutable config bundle (entities, fields, auth, flows) and **becomes** a full backend application at runtime. No per-app source code is generated. The same metadata is projected onto multiple protocols (REST, GraphQL, SSE, WebSocket), and all logic is expressed as no-code flows.

- No-code for the builder: apps are config + flows, never hand-written code.
- One interpreted engine, one config bundle (the Mendix model) — not multi-stack codegen.
- Modular monolith, one engine process per app.
- AI-ready: AI modules slot in as flow steps (uniform FlowStep interface).
- Multi-language via escape hatches (serverless, JS code-step, WASM) — not multiple engines.

2. High-Level Design

Request lifecycle:

```
HTTP (REST / GraphQL / SSE / WebSocket / Flow trigger)
-> Protocol surface -> AccessGuard (auth) -> DynamicCrudService <-> MetadataRegistry
-> SqlDialect -> parameterized SQL -> HikariCP -> Database
-> EntityEvent stream -> SSE / WebSocket subscribers
```

Startup: load bundle -> validate (JSON Schema + semantic) -> pick dialect -> build datasource -> sync schema (additive) -> expose enabled protocols -> live.

3. Low-Level Design — Modules

Module	Responsibility
config(.model)	Config POJOs; loader; JSON-Schema + semantic validation; MetadataRegistry
schema	SqlDialect (H2/PG/MySQL), Naming, SchemaManager (DDL sync)
data	DynamicCrudService (generic CRUD), FieldValidator, QueryOptions, IdempotencyStore
crypto / auth	AES-256-GCM EncryptionService; JWT + API-key + AccessGuard
flow(.steps)	FlowEngine; steps: set/condition/db/http/log/respond/ai/notify
audit / notify / ai	AuditService (audit log); NotificationService; AiProvider (stub + OpenAI-compatible)
realtime	EntityEvent, RealtimeService (SSE), WebSocket handler
api.*	REST, GraphQL, meta/OpenAPI, file, endpoint, flow, realtime surfaces
bootstrap	EngineConfiguration (composition root), EngineBootstrap, CORS

4. Field Types

Type	Notes
STRING/TEXT	length + validation (minLength/maxLength/pattern/email)
INT/LONG/DOUBLE/DECIMAL	numeric validation (min/max)
BOOLEAN / DATE / TIMESTAMP	auto createdAt/updatedAt by convention
JSON	native per dialect (JSONB/JSON)
REFERENCE	relation to another entity (app-level integrity)
FILE	upload/download/base64 via file API
encrypted:true	AES-256-GCM at rest, transparent on read

5. Application Flow (No-Code)

Flows = trigger + graph of typed steps over a shared context; steps read `#{input.x}`, `#{steps.id.field}`, `#{vars.x}`. The uniform FlowStep interface is where AI modules and connectors plug in.

```
flows:
  - name: grade
    steps:
      - {id: check, type: condition,
        params: {left: "#{input.score}", op: ">=", right: 50}, then: pass, else: fail}
      - {id: pass, type: respond, params: {body: {result: "pass"}}}
      - {id: fail, type: respond, params: {body: {result: "fail"}}}
# POST /flows/grade/run {"score":70} -> {"result":"pass"}
```

6. Protocols

Protocol	Status	Endpoint
REST	Done	/api/{entity}
GraphQL	Done	/graphql (+ /graphql/schema)
Server-Sent Events	Done	/realtime/stream
WebSocket	Done	/ws/events
Custom endpoints / webhooks	Done	/run/{name}, flow http step
Flow trigger	Done	/flows/{name}/run
GraphQL subscriptions / MCP	Roadmap P1	—
gRPC / OData	Roadmap P2	—
SOAP / Federation	Roadmap P3	—

7. Security

- Auth: JWT (/auth/login) or API-key (X-API-Key).
- Per-entity read/write role rules via one AccessGuard across all protocols.
- Field-level AES-256-GCM encryption; CORS; secrets via `#{ENV}`.
- Roadmap: row-level security, SSO/SAML/OIDC, audit log, multi-tenancy.

8. How to Run

```
cd flexforge
mvn spring-boot:run          # in-memory H2, zero setup
open http://localhost:8080/  # admin UI

# Postgres/MySQL (same image):
export DB_ENGINE=postgres DB_URL=jdbc:postgresql://localhost:5432/app \
      DB_USERNAME=app DB_PASSWORD=secret && mvn spring-boot:run

# different app, same engine:
export FLEXFORGE_CONFIG_PATH=/path/to/app.yaml && mvn spring-boot:run

mvn test                    # 26 integration tests on H2
mvn package                 # fat jar
mvn compile jib:build -Dimage=ghcr.io/you/flexforge-engine:0.1.0
```

9. Roadmap

- P0.5: select/multiselect + computed fields (formula/lookup/rollup), N:N relations.
- P1: AI steps + provider gateway + vector store; MCP protocol; serverless + JS code-step.
- P1.5: audit engine, row/field-level security, notifications, scheduler trigger.
- P2: visual builder, kanban/calendar/charts, connectors, multi-tenancy, gRPC/OData.
- P3: WASM plugins, vertical modules, SOAP, federation, offline.